Lava Architecture

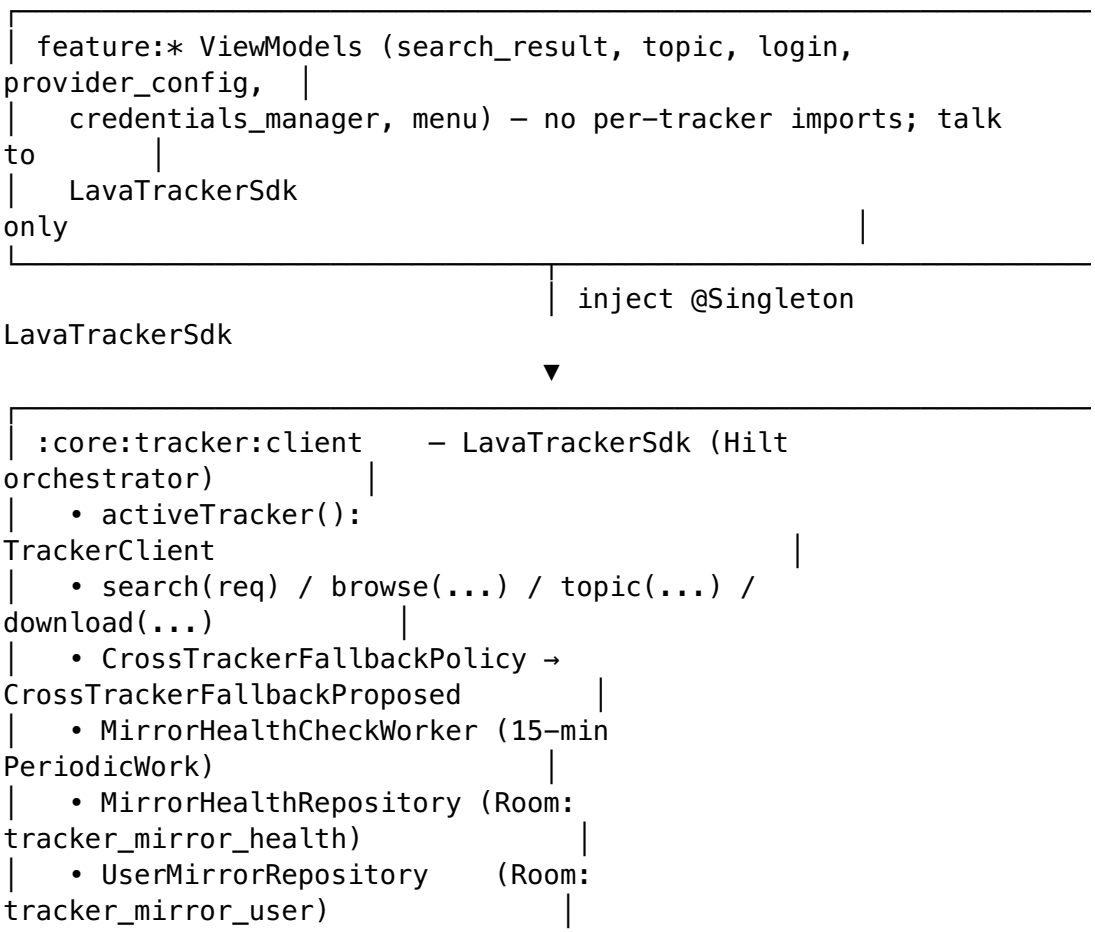
This document collects two architectural surfaces that span multiple modules:

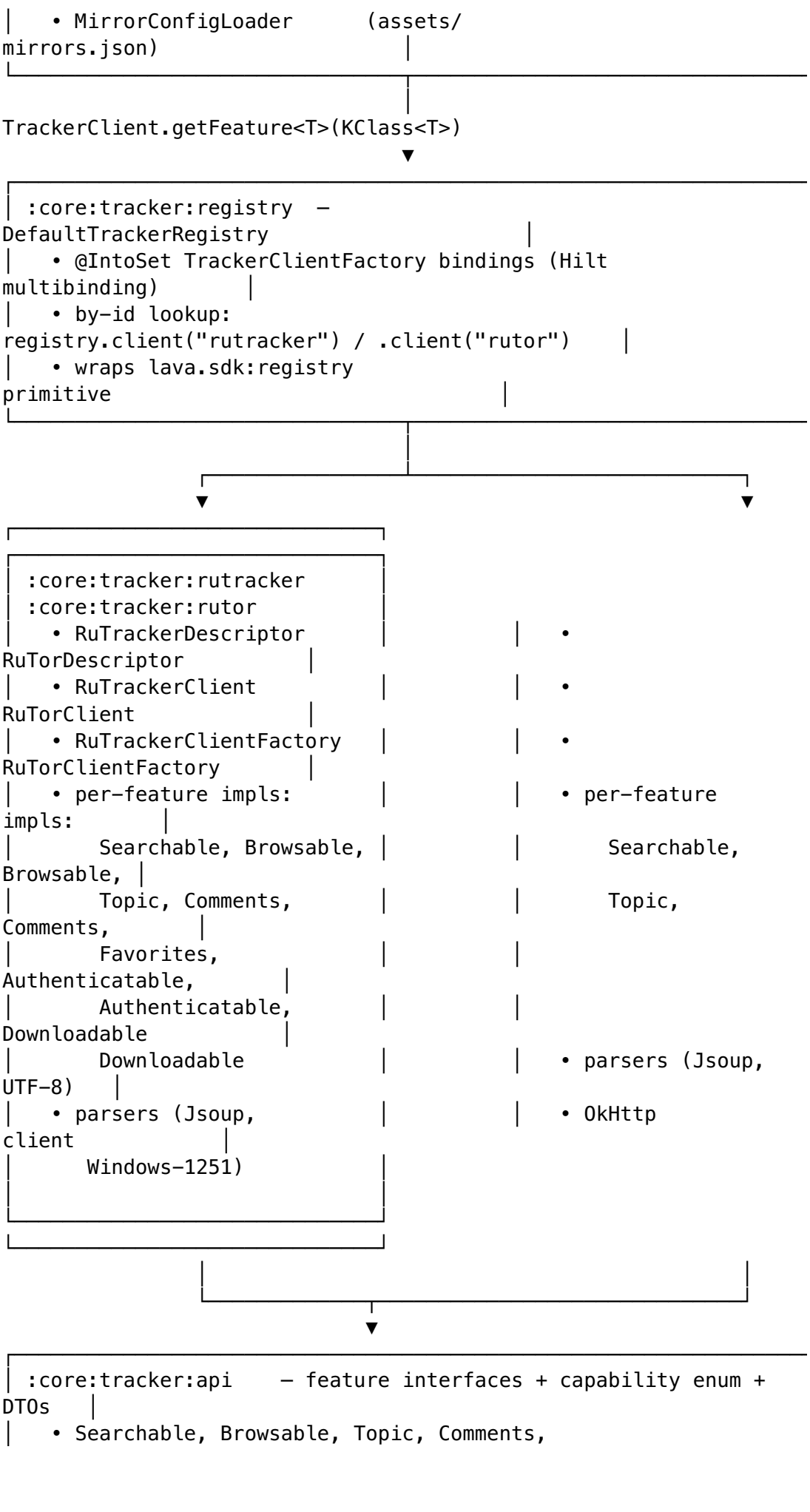
1. **Multi-Tracker SDK** — the layered, pluggable tracker stack added in SP-3a (1.2.0). Pure-Kotlin core, Hilt-injected orchestrator, per-tracker plugin modules, generic primitives in a `vasic-digital` submodule.
2. **Local Network Discovery** — the mDNS / NsdManager flow that auto-discovers the lava-api-go service on the LAN.

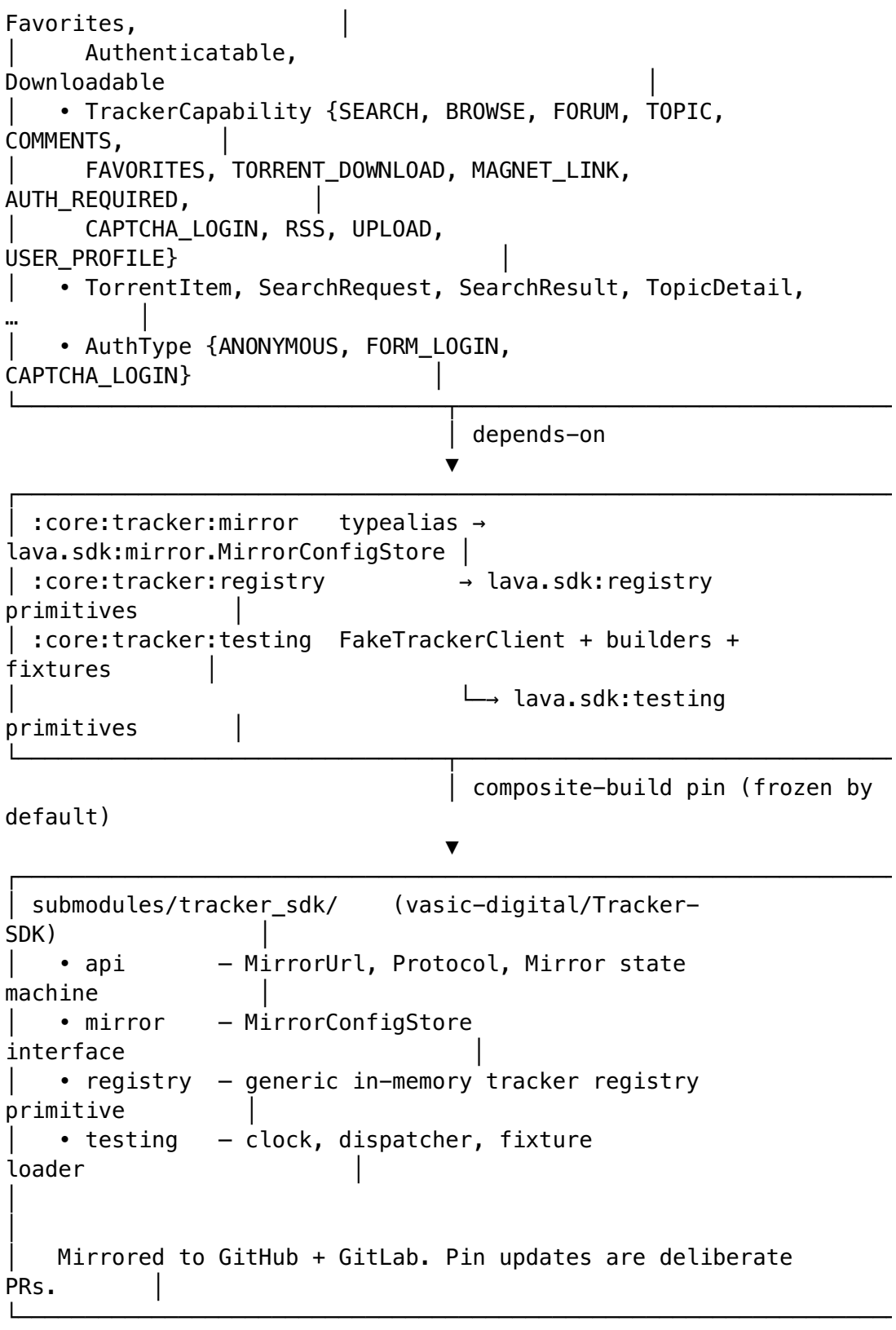
For per-feature MVI shapes, see the relevant `feature/*/README.md` (or the source — every feature mirrors `XxxState` + `XxxAction` + `XxxSideEffect` + `XxxViewModel`).

Multi-Tracker SDK (SP-3a, 1.2.0)

Layering

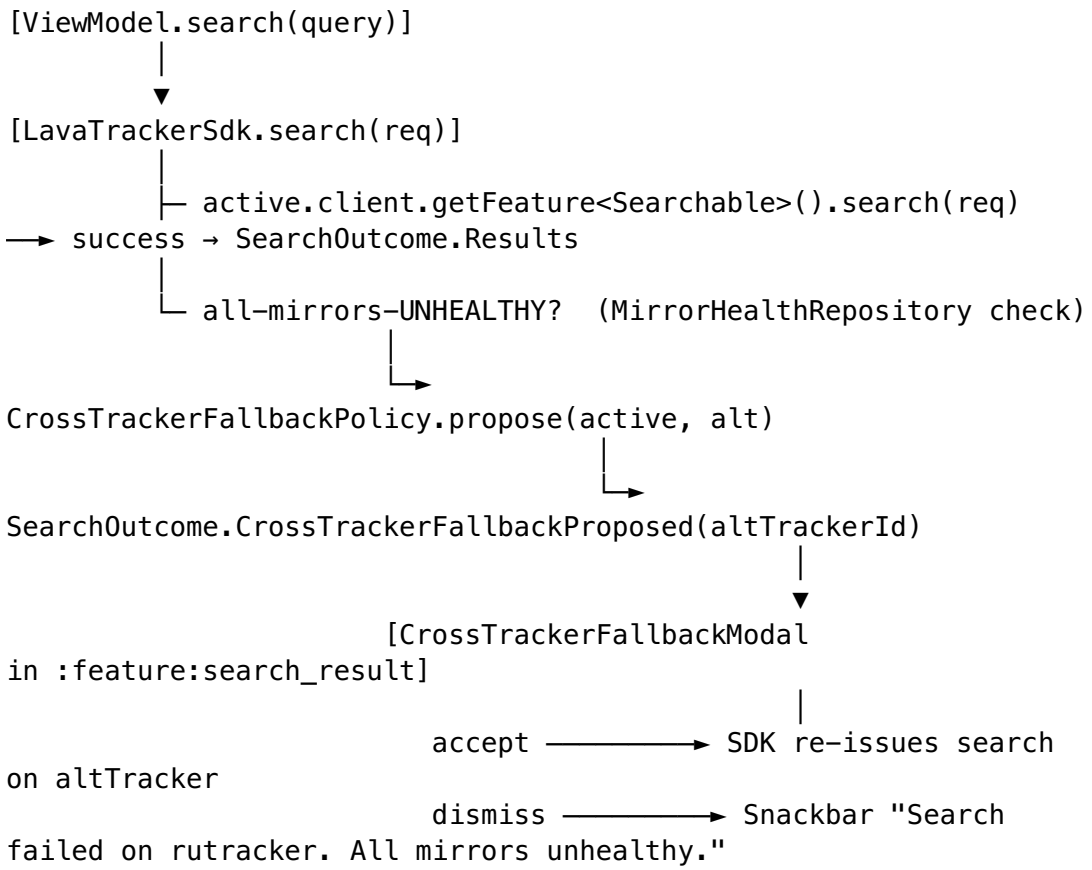



• MirrorConfigLoader (assets/ mirrors.json)
TrackerClient.getFeature<T>(KClass<T>)
:core:tracker:registry – DefaultTrackerRegistry
• @IntoSet TrackerClientFactory bindings (Hilt multibinding)
• by-id lookup: registry.client("rutracker") / .client("rutor")
• wraps lava.sdk:registry primitive
:core:tracker:rutracker
:core:tracker:rutor
• RuTrackerDescriptor
• RuTorDescriptor
• RuTrackerClient
• RuTorClient
• RuTrackerClientFactory
• RuTorClientFactory
• per-feature impls: Searchable, Browsable, Topic, Comments, Favorites, Authenticatable, Downloadable
• per-feature impls: Searchable, Browsable, Topic, Comments, Authenticatable, Downloadable
• parsers (Jsoup, Windows-1251)
• parsers (Jsoup, UTF-8)
• OkHttp client
:core:tracker:api – feature interfaces + capability enum + DTOs
• Searchable, Browsable, Topic, Comments,



The capability enum is the single source of truth. Constitutional clause 6.E (Capability Honesty) requires that **every capability declared in a `TrackerDescriptor` resolves to a non-null `TrackerClient.getFeature<T>()`**; the `:core:tracker:registry` test enumerates descriptors + capabilities and asserts non-null at JVM-test time.

Cross-tracker fallback flow



There is **no silent fallback**. The user is always shown the modal and must explicitly accept or dismiss.

Persistence

Room database `AppDatabase` v7 owns two SP-3a tables (migration v6 → v7):

Table	Entity	Purpose
`tracker_mirror_health`	`MirrorHealthEntity`	Per-mirror probe results (HEALTHY/DEGRADED/UNHEALTHY)
`tracker_mirror_user`	`UserMirrorEntity`	User-added custom mirrors per tracker

Bundled defaults live at `core/tracker/client/src/main/assets/mirrors.json` and are merged at app start by `MirrorConfigLoader`.

Module reference

Path	Plugin	Purpose
`core/tracker/api`	`lava.kotlin.library` + serialization	Public feature interfaces +

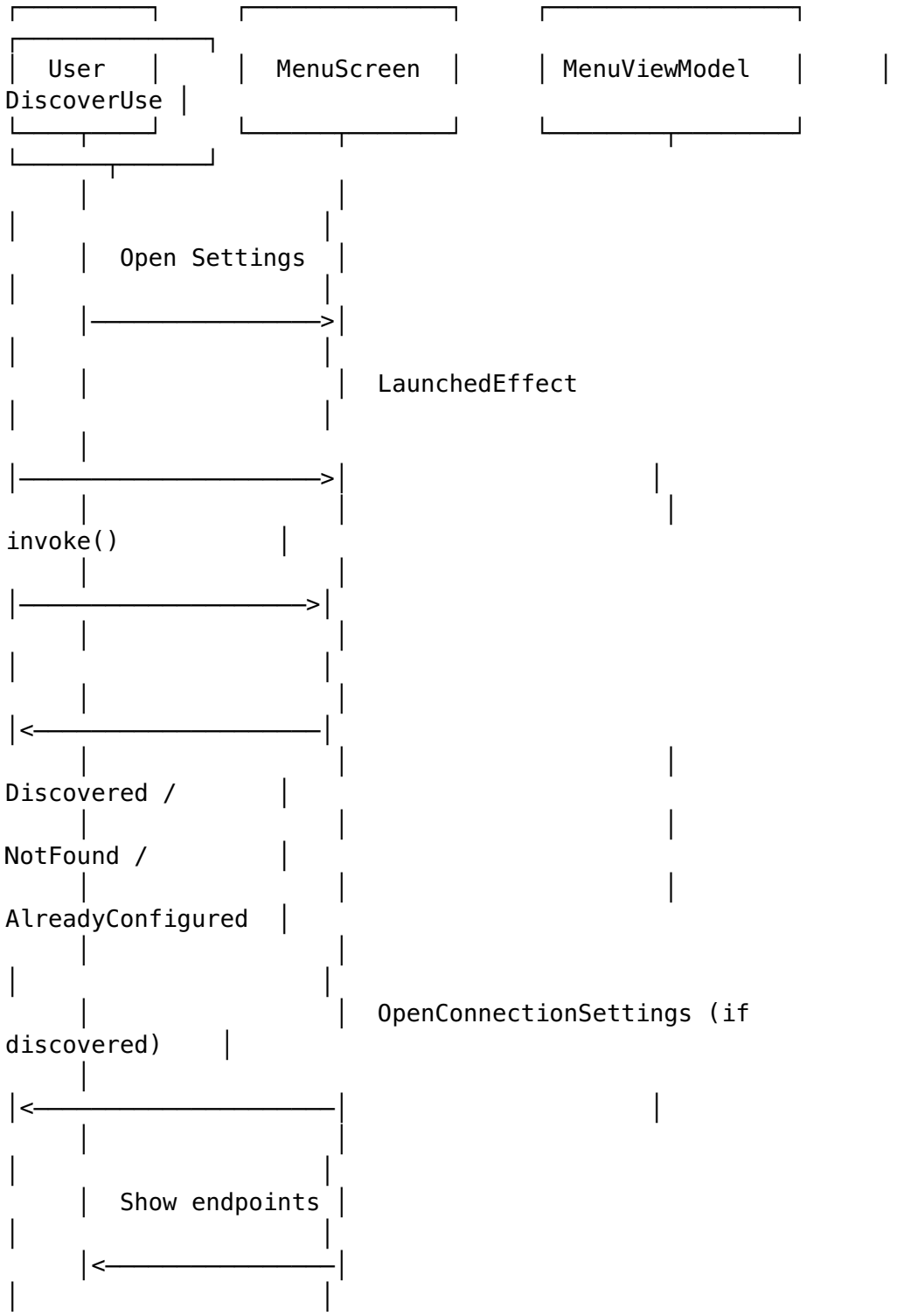
Path	Plugin	Purpose
		capability enum + DTOs
`core/tracker/registry`	`lava.kotlin.library`	Lava-domain `TrackerRegistry` over `lava.sdk:registry`
`core/tracker/mirror`	`lava.kotlin.library` + serialization	`MirrorConfigStore` typealias bridge
`core/tracker/client`	`lava.android.library` + Hilt + serialization	`LavaTrackerSdk`, persistence repos, WorkManager
`core/tracker/rutracker`	`lava.kotlin.tracker.module`	RuTracker plugin
`core/tracker/rutor`	`lava.kotlin.tracker.module`	RuTor plugin
`core/tracker/testing`	`lava.kotlin.library`	`FakeTrackerClient` + builders + fixture loaders
`feature/provider_config`	`lava.android.feature` + Compose	Per-provider configuration (mirrors, credentials binding, sync toggle, anonymous mode, clone). Reachable from Menu by tapping a provider row. Replaces the SP-3a Trackers screen (deleted in SP-4 Phase C).
`feature/credentials_manager`	`lava.android.feature` + Compose	Passphrase-gated credentials CRUD UI (Phase A). Add / edit / delete `CredentialsEntry` rows.

Generic primitives mounted via composite build at `submodules/tracker_sdk/` (submodule pin frozen by default per the Decoupled Reusable Architecture constitutional rule).

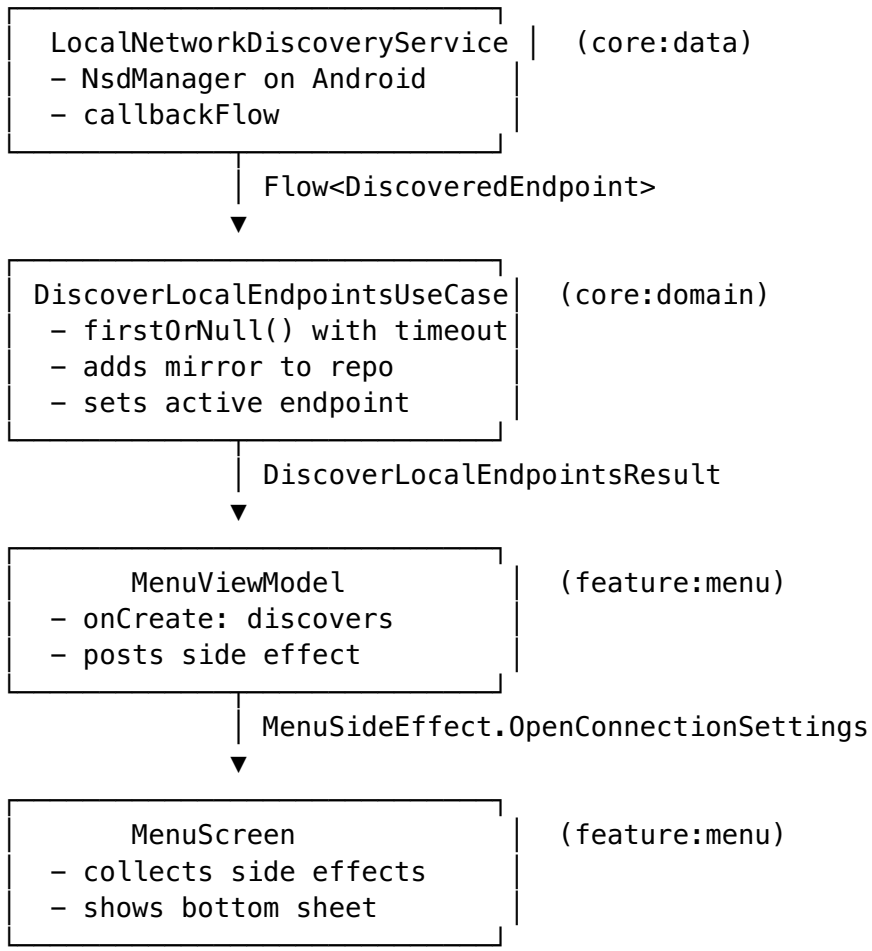
For the full design rationale see `docs/superpowers/specs/2026-04-30-sp3a-multi-tracker-sdk-foundation-design.md`. For the step-by-step recipe to add a third tracker see `docs/sdk-developer-guide.md`.

Local Network Discovery Architecture

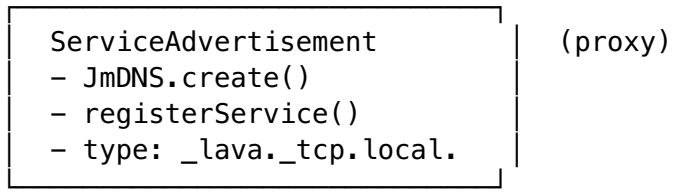
Sequence Diagram



Data Flow



Proxy Advertisement



Module Dependencies

```
feature:menu
    └── core:domain
            ├── core:data
            │       └── LocalNetworkDiscoveryService (NsdManager)
            └── core:models

proxy
    └── JmDNS advertisement
```

On-Device Lava API (Android) — third app + SQLite backend

A third artifact is being added alongside `:app` (the client) and the host `lava-api-go` server: a standalone **Lava API Android app** (`:api-app`, applicationId `digital.vasic.lava.api`, debug suffix `.dev`) that boots the full Lava API **in-process** on a phone/tablet, backed by a local SQLite database, and exposes it on the LAN via mDNS.

Landed (Phases A + B):

- **Additive SQLite storage backend** in `lava-api-go`. A `storage.Storage` interface (`Get`/`Set`/`Invalidate`/`Close`) with two interchangeable implementations — the existing Postgres adapter (unchanged) and a new pure-Go `modernc.org/sqlite` impl (WAL, INCREMENTAL auto_vacuum, 10-min background GC, lazy TTL expiry). Selected at startup by `LAVA_API_STORAGE_BACKEND` (`postgres` default | `sqlite`). The Postgres default behaves byte-for-byte as before.
- **In-process Go embed** (`internal/mobile`): `Start`/`Stop`/`Status` boot the EXACT production Gin router (shared `internal/router.Build`) over HTTP/1.1+H2 TLS, bound to `0.0.0.0`, SQLite-backed, enforcing the identical Lava-Auth gate the host binary uses. Cross-compiled for Android via `go build -buildmode=c-shared` (gomobile is blocked by the relative `replace ../submodules/*` directives) plus a hand-written JNI bridge exposing `digital.vasic.lava.apigo.LavaNative` (`nativeStart`/`Stop`/`Status`).

Pending (Phases C/D/E): the `:core:apiengine` Kotlin wrapper, the `:api-app` Compose module, the foreground `ApiEngineService`, the on-device `NsdManager` advertiser (TXT `engine=go, platform=android, storage=sqlite`), the landing UI, and the instrumented Challenge tests.

Full detail and Mermaid diagrams (component, lifecycle, mDNS+auth sequence, build pipeline): `docs/ON_DEVICE_API.md`. User-facing guide: `docs/guides/ON_DEVICE_API_USER_GUIDE.md`.